

```

import re

#
=====
=====
# Knowledge Base (unchanged from Week 6)
#
=====
=====
knowledge_base = {
    "locations_hours": {"patient_text":
        "HealthConnect Clinic has two locations:
        Central Clinic at 14 Wellness Avenue, and
        Lakeside Clinic at 8 Care Street. Open Mon-
        Fri 8AM-6PM, Sat 9AM-2PM.",
        "internal_note": None},
    "services": {"patient_text":
        "HealthConnect offers general outpatient
        consultations, follow-up consultations,
        selected specialist consultations by
        appointment, diagnostic and lab services by
        referral, and preventive wellness
        consultations.", "internal_note": "Do not
        determine whether a patient needs a
        particular service.", "known_subtopics":
        ["general", "follow-up", "specialist (by
        appointment)", "diagnostic/lab (by
        referral)", "preventive/wellness"]},
    "booking": {"patient_text": "You can
        request an appointment through the booking

```

```
channel, reception, or an approved
platform. Select location, date and time,
provide contact info, review before
confirming, and keep your reference.",
"internal_note": None, "known_subtopics":
["standard advance booking"]},
    "rescheduling": {"patient_text":
"Contact the clinic as early as possible to
reschedule or cancel. Include your
appointment reference. Rescheduling is
subject to availability.", "internal_note":
None},
    "late_arrival": {"patient_text":
"Arrive at least 15 minutes early. Late
arrivals may need to wait or be
rescheduled. Contact reception if expecting
to be late.", "internal_note": None},
    "what_to_bring": {"patient_text":
"Bring valid ID where required, your
appointment reference, and any relevant
referral documents.", "internal_note": "Do
not give personalised clinical preparation
instructions."},
    "billing": {"patient_text": "Payment
requirements vary by service. Contact
reception for accepted payment methods and
estimated charges.", "internal_note": "Do
not invent prices or insurance coverage."},
}

emergency_keywords = ["chest pain", "chest
hurts", "can't breathe", "cant breathe",
"cannot breathe", "difficulty breathing",
"trouble breathing", "bleeding heavily",
```

```
"unconscious", "collapsed", "severe pain",
"emergency", "dying", "suicidal",
"overdose", "stroke", "heart attack"]
medical_keywords = ["diagnose",
"diagnosis", "what's wrong with me", "whats
wrong with me", "what is wrong with me",
"symptom", "symptoms", "headache", "fever",
"pain", "sick", "illness", "medication",
"medicine", "prescribe", "prescription",
"treatment", "dosage", "is it serious", "do
i have"]
topic_keywords = {
    "locations_hours": ["open", "hours",
"time", "location", "address", "where",
"saturday", "sunday", "close"],
    "services": ["service", "offer",
"consultation", "specialist", "lab",
"wellness", "checkup", "pediatric",
"paediatric"],
    "booking": ["book", "appointment",
"schedule", "reserve", "how do i book"],
    "rescheduling": ["reschedule",
"cancel", "cancellation", "change my
appointment", "move my appointment"],
    "late_arrival": ["late", "arrive",
"arrival", "missed", "no-show", "no show"],
    "what_to_bring": ["bring", "need to
bring", "what should i bring", "id",
"identification", "documents"],
    "billing": ["price", "cost", "pay",
"payment", "insurance", "bill", "billing",
"how much"],
}
reminder_keywords = ["reminder", "remind
```

```

me", "which reminder", "best way to
remind", "notify me"]
REMINDER_TIP = ("Based on HealthConnect's
own appointment data, patients reminded by
SMS had the "
    "lowest no-show rate of
any channel tested (45.8%, vs. 51.4% with
no reminder at all) "
    "- so it's worth
confirming your preferred reminder channel
when you book.")

# NEW in Week 7: phrases that ask the
assistant to PERFORM an action,
# not just retrieve information (fixes the
"please confirm my appointment
# right now" bug found in Week 5 and
confirmed again in Week 6)
action_request_keywords = [
    "book this", "book it", "book my",
    "book me", "confirm my appointment",
    "make the appointment", "reserve me",
    "schedule me", "set up my appointment",
    "do it for me", "right now", "for me
now",
]

# NEW in Week 7: specific
service/availability questions the
Knowledge Base
# does not explicitly confirm or deny
(fixes the "generic non-answer" bug
# found in Week 6 testing)
unconfirmed_specifics = {

```

```
    "pediatric": "pediatric or child-  
specific consultations",  
    "paediatric": "pediatric or child-  
specific consultations",  
    "same-day": "same-day appointment  
availability",  
    "same day": "same-day appointment  
availability",  
    "emergency slot": "emergency/walk-in  
appointment slots",  
    "walk-in": "walk-in appointments",  
    "walk in": "walk-in appointments",  
}
```

```
def classify(query):  
    q = query.lower()  
    for kw in emergency_keywords:  
        if kw in q: return "emergency"  
    for kw in medical_keywords:  
        if kw in q: return "medical"  
    return "in_scope_candidate"
```

```
def is_action_request(query):  
    q = query.lower()  
    return any(kw in q for kw in  
action_request_keywords)
```

```
def find_unconfirmed_specific(query):  
    q = query.lower()  
    for kw, label in  
unconfirmed_specifics.items():  
        if kw in q:  
            return label  
    return None
```

```

def retrieve(query):
    q = query.lower()
    scores = {t: sum(1 for kw in kws if kw
in q) for t, kws in topic_keywords.items()}
    scores = {k: v for k, v in
scores.items() if v > 0}
    return max(scores, key=scores.get) if
scores else None

def respond_week6(query):
    """Reproduced for before/after
comparison only."""
    category = classify(query)
    if category == "emergency":
        return {"response": "This sounds
like it could be a medical emergency.
Please seek immediate help from the
appropriate emergency service or your
nearest emergency facility right away."}
    elif category == "medical":
        return {"response": "I'm not able
to assess symptoms, diagnose conditions, or
give medical advice. I'd recommend speaking
with a qualified healthcare professional."}
    q = query.lower()
    is_reminder_question = any(kw in q for
kw in reminder_keywords)
    topic = retrieve(query)
    if topic is None:
        if is_reminder_question:
            return {"response": "Reminder
preferences aren't something I can set for

```

```

you directly, but here's something useful:
" + REMINDER_TIP}

        return {"response": "I don't have
information on that. I'd recommend
contacting clinic reception directly."}
        response = "Here's what I can tell you:
" + knowledge_base[topic]["patient_text"]
        if topic == "booking" or
is_reminder_question:
            response += " " + REMINDER_TIP
        return {"response": response}

def respond_week7(query):
    category = classify(query)
    if category == "emergency":
        return {"query": query, "response":
"This sounds like it could be a medical
emergency. Please seek immediate help from
the appropriate emergency service or your
nearest emergency facility right away."}
    elif category == "medical":
        return {"query": query, "response":
"I'm not able to assess symptoms, diagnose
conditions, or give medical advice. I'd
recommend speaking with a qualified
healthcare professional."}

    q = query.lower()

    # FIX 1 (Week 7): catch action requests
before topic retrieval,
    # regardless of which topic they'd
otherwise match

```

```

        if is_action_request(query):
            return {"query": query, "response":
                "I'm not able to book, confirm, or make
                changes to appointments directly – I can
                only provide information. To actually book
                or confirm an appointment, please contact
                clinic reception or use the approved
                booking platform."}

        is_reminder_question = any(kw in q for
                                     kw in reminder_keywords)

        # FIX 2 (Week 7): explicitly flag
        # specific questions the Knowledge Base
        # doesn't confirm or deny, instead of
        # silently giving a generic answer
        unconfirmed =
        find_unconfirmed_specific(query)
        if unconfirmed:
            return {"query": query, "response":
                f"I don't have specific confirmation about
                {unconfirmed} in what I'm approved to
                answer from. I'd recommend contacting
                clinic reception directly to check this."}

        topic = retrieve(query)
        if topic is None:
            if is_reminder_question:
                return {"query": query,
                    "response": "Reminder preferences aren't
                    something I can set for you directly, but
                    here's something useful: " + REMINDER_TIP}
            return {"query": query, "response":
                "I don't have information on that. I'd

```



```

recommend contacting clinic reception
directly."}

    response = "Here's what I can tell you:
" + knowledge_base[topic]["patient_text"]
    if topic == "booking" or
is_reminder_question:
        response += " " + REMINDER_TIP
    return {"query": query, "response":
response}

if __name__ == "__main__":
    print("=" * 70)
    print("BEFORE / AFTER – Fix 1: Action
vs Information requests")
    print("=" * 70)
    for q in ["Can you book this
appointment for me right now?", "Please
confirm my appointment right now"]:
        print(f"\nQ: {q}")
        print(f"  BEFORE (Week 6):
{respond_week6(q)['response']}")
        print(f"  AFTER  (Week 7):
{respond_week7(q)['response']}")

    print("\n\n" + "=" * 70)
    print("BEFORE / AFTER – Fix 2:
Unconfirmed specific questions")
    print("=" * 70)
    for q in ["Do you do pediatric
consultations?", "Can I get a same-day
appointment?"]:
        print(f"\nQ: {q}")

```

```
        print(f" BEFORE (Week 6):  
{respond_week6(q)['response']}")  
        print(f" AFTER  (Week 7):  
{respond_week7(q)['response']}")
```